Project Iteration 3 Grading Rubric

1. One or more UML **Class**,**Sequence** and **State Machine** diagrams showing your system. The class diagram should be consistent with the sequence diagram and all names should match between the two. The Sequence Diagram ONLY has to show interactions between processes, internal details within a thread are not needed. Class and method names in the source code should match class, attribute and association names in the diagrams.

GA 5.1	1	2	3	4
20%	Produces poor diagrams and/or sketches which are difficult to interpret. **No Readme file.**	Produces diagrams and/or sketches which are incomplete, without proper labelling or descriptors.	Constructs diagrams and/or sketches which adequately convey the desired concept.	Creates diagrams and/or sketches which clearly convey the desired concept and provide sufficient descriptors such that all information needed to interpret the diagrams and/or sketches is provided

2. The source code for all three parts of the system, as well as any files required to run these files in Eclipse (Note to TA's.. part marks are acceptable here, especially for categories 3 and 4.)

	1	2	3	4
60%	The code for all three tasks is present and compiles.	The code runs but not on three machines at the same time (i.e., the test can run on one machine, but three distinct processes are required).	The code for all three tasks runs perfectly as three separate processes (meets specifications), however the style of the code is lacking in that names are not meaningful, code is not indented, code is overly complex or tricky in places, or it is poorly commented.	The code runs perfectly **and is well written** (single responsibility principle - each method does one thing, etc) **and is easy to read**. All classes are documented and **meaningful comments** throughout as needed. Use of JavaDoc is encouraged.

Project Iteration 3 Grading Rubric

3. Testing

4.3	1	2	3	4
20%	Testing involves running the final submission only. The code “runs”.	Some unit tests are present. Some attempt is made at exercising the state machine. **(or NO source code supplied for unit tests).**	Unit tests for the state machine only. An attempt is made to exercise all the states.	Unit tests for all classes are conducted using a testing framework such as Junit. The state machine and schedulers are exercised.